📚 Training Guide — Part 4

Chapters 15 · 16 · 17

Inheritance & Polymorphism · Overloading · Virtual Functions

```
╔═══════════════════════════════════════════════════════════════════╗
║       C++ ADVANCED OOP — COMPLETE GUIDE (using iostream)          ║
║                                                                   ║
║   15.0  Inheritance and Polymorphism                              ║
║   16.0  Function Overloading and Operator Overloading             ║
║   17.0  Virtual Functions, Binding, Abstract Classes              ║
╚═══════════════════════════════════════════════════════════════════╝
```

> **Note on style:** All examples in this guide use `#include <iostream>` and `std::cout` / `std::cin` instead of `printf` / `scanf`. This is idiomatic modern C++.

📋 Table of Contents

15.0 Inheritance and Polymorphism

🎯 Learning Objectives

- Master all four types of inheritance: simple, multiple, multilevel, hierarchical
- Understand how constructors and destructors behave in an inheritance chain
- Distinguish public, protected, and private inheritance and when to use each
- Implement both static and dynamic polymorphism with concrete examples
- Understand the role of `virtual` in enabling runtime dispatch

Introduction: Standing on the Shoulders of Giants

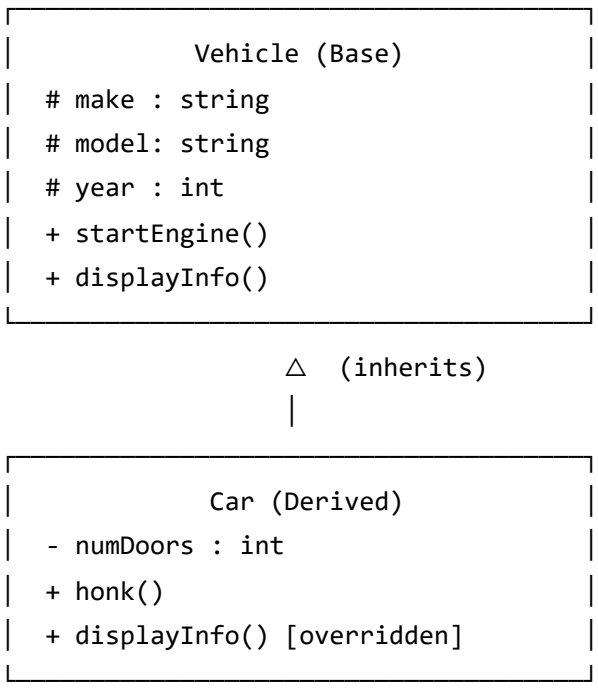
Inheritance is one of OOP's most powerful features: it lets you **build new classes from existing ones**, inheriting all tested, working behavior and extending it with new capabilities. You write less code, reuse more, and model real-world hierarchies naturally.

```
WITHOUT INHERITANCE (duplicate code everywhere):
_____________________________________________________

class Dog   { string name; int age; void eat(); void sleep(); void bark(); };
class Cat   { string name; int age; void eat(); void sleep(); void meow(); };
class Bird  { string name; int age; void eat(); void sleep(); void fly(); };
// eat() and sleep() are copy-pasted THREE times. Fix a bug once = fix it everywhere!

WITH INHERITANCE (write once, reuse always):
_____________________________________________

class Animal { string name; int age; void eat(); void sleep(); };
class Dog  : public Animal { void bark(); };   // inherits eat, sleep
class Cat  : public Animal { void meow(); };   // inherits eat, sleep
class Bird : public Animal { void fly(); };    // inherits eat, sleep
// eat() and sleep() written ONCE. A bug fix propagates to all three.
```

15.1 Introduction to Inheritance

Simple Inheritance

Simple (single) inheritance: one child class inherits from exactly one parent class. This is the most common form.


Vehicle (Base)
make : string
model: string
year : int
+ startEngine()
+ displayInfo()
△ (inherits)
Car (Derived)
- numDoors : int
+ honk()
+ displayInfo() [overridden]

```cpp
#include <iostream>
#include <string>

// ─── BASE CLASS ──────────────────────────────────────────────
class Vehicle {
protected:                      // 'protected': child classes can access
    std::string make;
    std::string model;
    int         year;
    double      speed;          // current speed in km/h

public:
    // Constructor
    Vehicle(const std::string& mk, const std::string& mdl, int yr)
        : make(mk), model(mdl), year(yr), speed(0.0) {
        std::cout << "[Vehicle] Constructed: "
                  << year << " " << make << " " << model << "\n";
    }

    // Virtual destructor — ALWAYS needed in a base class!
    virtual ~Vehicle() {
        std::cout << "[Vehicle] Destroyed: "
                  << year << " " << make << " " << model << "\n";
    }

    void startEngine() const {
        std::cout << make << " " << model << " engine started. Vroom!\n";
    }

    void accelerate(double kmh) {
        speed += kmh;
        std::cout << make << " " << model
                  << " accelerating. Speed: " << speed << " km/h\n";
    }

    void brake() {
        speed = (speed > 20.0) ? speed - 20.0 : 0.0;
        std::cout << make << " " << model
                  << " braking. Speed: " << speed << " km/h\n";
    }

    // virtual: child classes may override this
    virtual void displayInfo() const {
```

```
        std::cout << "Vehicle: " << year << " " << make
                  << " " << model << " | Speed: " << speed << " km/h\n";
    }

    // Getters
    std::string getMake()  const { return make; }
    std::string getModel() const { return model; }
    int         getYear()  const { return year; }
};

// ─── DERIVED CLASS ──────────────────────────────────────────────
class Car : public Vehicle {       // 'public' inheritance (most common)
private:
    int    numDoors;
    bool   sunroof;
    double fuelLevel;   // 0.0 to 100.0 percent

public:
    // Child constructor MUST call parent constructor via initializer list
    Car(const std::string& mk, const std::string& mdl, int yr,
        int doors, bool hasSunroof)
        : Vehicle(mk, mdl, yr),    // ← Parent constructor called first
          numDoors(doors),
          sunroof(hasSunroof),
          fuelLevel(100.0)
    {
        std::cout << "[Car] Constructed: "
                  << doors << "-door"
                  << (hasSunroof ? " with sunroof" : "") << "\n";
    }

    ~Car() override {
        std::cout << "[Car] Destroyed: " << make << " " << model << "\n";
    }

    void honk() const {
        std::cout << make << " " << model << ": Beep beep!\n";
    }

    void refuel(double amount) {
        fuelLevel = std::min(fuelLevel + amount, 100.0);
        std::cout << "Refueled. Fuel level: " << fuelLevel << "%\n";
    }
```

```
    // OVERRIDE parent's displayInfo
    void displayInfo() const override {
        std::cout << "Car:     " << year << " " << make << " " << model
                  << " | " << numDoors << " doors"
                  << (sunroof ? " | sunroof" : "")
                  << " | Fuel: " << fuelLevel << "%"
                  << " | Speed: " << speed << " km/h\n";
    }
};

int main() {
    std::cout << "=== Simple Inheritance Demo ===\n\n";

    std::cout << "--- Creating Car ---\n";
    Car myCar("Toyota", "Camry", 2023, 4, true);

    std::cout << "\n--- Using inherited methods ---\n";
    myCar.startEngine();      // Inherited from Vehicle
    myCar.accelerate(60.0);   // Inherited from Vehicle
    myCar.accelerate(40.0);   // Inherited from Vehicle

    std::cout << "\n--- Using Car-specific methods ---\n";
    myCar.honk();             // Car-specific

    std::cout << "\n--- Display (overridden) ---\n";
    myCar.displayInfo();      // Car's version (overridden)

    std::cout << "\n--- Scope ends — destructors called ---\n";
    // Destruction order: Child first, then Parent
    return 0;
}
```

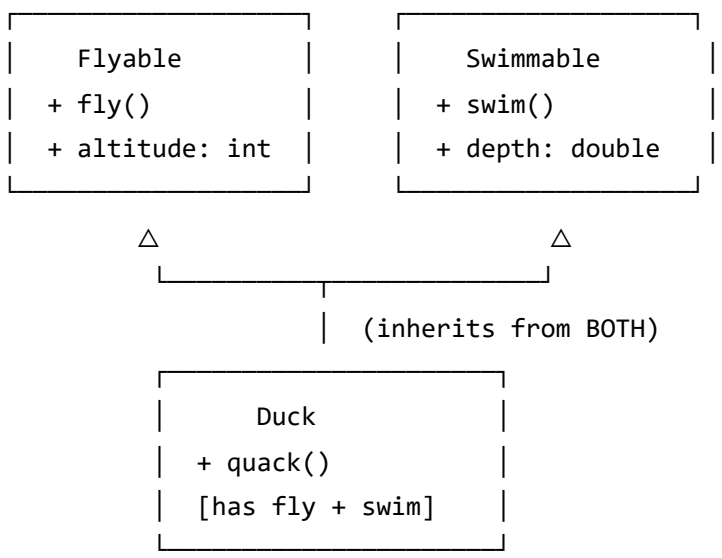
Output:

```
=== Simple Inheritance Demo ===

--- Creating Car ---
[Vehicle] Constructed: 2023 Toyota Camry
[Car] Constructed: 4-door with sunroof

--- Using inherited methods ---
Toyota Camry engine started. Vroom!
Toyota Camry accelerating. Speed: 60 km/h
Toyota Camry accelerating. Speed: 100 km/h

--- Using Car-specific methods ---
Toyota Camry: Beep beep!

--- Display (overridden) ---
Car:     2023 Toyota Camry | 4 doors | sunroof | Fuel: 100% | Speed: 100 km/h

--- Scope ends — destructors called ---
[Car] Destroyed: Toyota Camry
[Vehicle] Destroyed: 2023 Toyota Camry
```

> 💡 **Key Observation:** Constructor order is **Parent first, then Child**. Destructor order is **Child first, then Parent**. Always.

Multiple Inheritance

Multiple inheritance: a child class inherits from **two or more** parent classes simultaneously.


Flyable
+ fly()
+ altitude: int
Swimmable
+ swim()
+ depth: double
(inherits from BOTH)
Duck
+ quack()
[has fly + swim]

```cpp
#include <iostream>
#include <string>

class Flyable {
protected:
    double altitude;   // metres above ground
public:
    Flyable() : altitude(0.0) {}

    void fly(double metres) {
        altitude += metres;
        std::cout << "Flying at " << altitude << " metres altitude.\n";
    }

    void land() {
        altitude = 0.0;
        std::cout << "Landed safely.\n";
    }

    double getAltitude() const { return altitude; }
};

class Swimmable {
protected:
    double depth;      // metres below surface
public:
    Swimmable() : depth(0.0) {}

    void swim(double metres) {
        depth += metres;
        std::cout << "Swimming at " << depth << " metres depth.\n";
    }

    void surface() {
        depth = 0.0;
        std::cout << "Surfaced.\n";
    }

    double getDepth() const { return depth; }
};

// ─── MULTIPLE INHERITANCE: Duck IS-A Flyable AND IS-A Swimmable ──────
class Duck : public Flyable, public Swimmable {
```

```
private:
    std::string name;
public:
    Duck(const std::string& n) : Flyable(), Swimmable(), name(n) {
        std::cout << "Duck '" << name << "' created.\n";
    }

    void quack() const {
        std::cout << name << ": Quack quack!\n";
    }

    void showStatus() const {
        std::cout << name << " | altitude: " << altitude
                  << "m | depth: " << depth << "m\n";
    }
};

int main() {
    std::cout << "=== Multiple Inheritance Demo ===\n\n";

    Duck donald("Donald");
    donald.quack();

    std::cout << "\n";
    donald.fly(50.0);
    donald.fly(30.0);
    donald.land();

    std::cout << "\n";
    donald.swim(5.0);
    donald.swim(3.0);
    donald.surface();

    std::cout << "\n";
    donald.showStatus();

    return 0;
}
```

Multilevel Inheritance

Multilevel inheritance: A inherits B, B inherits C — forming a chain. Each level adds specialization.

```
Animal  (level 1 – most general)
   |
Mammal  (level 2 – more specific)
   |
Dog     (level 3 – most specific)
   |
GoldenRetriever  (level 4)
```

```cpp
#include <iostream>
#include <string>

// ─── LEVEL 1: Most general ──────────────────────────────────────
class LivingThing {
protected:
    std::string name;
    int         age;
public:
    LivingThing(const std::string& n, int a) : name(n), age(a) {
        std::cout << "[LivingThing] " << name << " is alive.\n";
    }
    virtual ~LivingThing() {
        std::cout << "[LivingThing] " << name << " lifecycle ended.\n";
    }
    void breathe() const {
        std::cout << name << " breathes.\n";
    }
    void grow() const {
        std::cout << name << " grows. Age: " << age << "\n";
    }
};

// ─── LEVEL 2: More specific ─────────────────────────────────────
class Animal : public LivingThing {
protected:
    bool canMove;
public:
    Animal(const std::string& n, int a, bool moves)
        : LivingThing(n, a), canMove(moves) {
        std::cout << "[Animal] " << name
                  << (canMove ? " can move." : " cannot move.") << "\n";
    }
    virtual ~Animal() {
        std::cout << "[Animal] " << name << " animal lifecycle ended.\n";
    }
    void eat() const {
        std::cout << name << " eats food.\n";
    }
    virtual void makeSound() const {
        std::cout << name << " makes a sound.\n";
    }
};
```

```cpp
// ── LEVEL 3: Even more specific ──────────────────────────────
class Mammal : public Animal {
protected:
    bool isWarmBlooded;
    int  numLegs;
public:
    Mammal(const std::string& n, int a, int legs)
        : Animal(n, a, true), isWarmBlooded(true), numLegs(legs) {
        std::cout << "[Mammal] " << name << " | legs: "
                  << numLegs << " | warm-blooded\n";
    }
    virtual ~Mammal() {
        std::cout << "[Mammal] " << name << " mammal lifecycle ended.\n";
    }
    void nurseYoung() const {
        std::cout << name << " nurses its young with milk.\n";
    }
};

// ── LEVEL 4: Most specific ──────────────────────────────────
class Dog : public Mammal {
private:
    std::string breed;
    bool        isTrained;
public:
    Dog(const std::string& n, int a, const std::string& br, bool trained)
        : Mammal(n, a, 4), breed(br), isTrained(trained) {
        std::cout << "[Dog] Breed: " << breed
                  << (isTrained ? " | trained\n" : " | untrained\n");
    }
    ~Dog() override {
        std::cout << "[Dog] " << name << " dog lifecycle ended.\n";
    }

    // Override inherited virtual
    void makeSound() const override {
        std::cout << name << " says: Woof woof!\n";
    }

    void fetch() const {
        std::cout << name << " fetches the ball!\n";
    }
```

```
    void showPedigree() const {
        std::cout << "Name: " << name
                  << " | Breed: " << breed
                  << " | Age: " << age << " years"
                  << " | Trained: " << (isTrained ? "Yes" : "No") << "\n";
    }
};

int main() {
    std::cout << "=== Multilevel Inheritance Demo ===\n\n";

    std::cout << "--- Construction (bottom-up in initializer list, "
                 "top-down in execution) ---\n";
    Dog rex("Rex", 3, "German Shepherd", true);

    std::cout << "\n--- Using methods from every level ---\n";
    rex.breathe();       // LivingThing
    rex.grow();          // LivingThing
    rex.eat();           // Animal
    rex.makeSound();     // Dog (overridden)
    rex.nurseYoung();    // Mammal
    rex.fetch();         // Dog
    rex.showPedigree();  // Dog

    std::cout << "\n--- Destruction (child first, then up the chain) ---\n";
    return 0;
}
```

Output:

```
=== Multilevel Inheritance Demo ===

--- Construction ---
[LivingThing] Rex is alive.
[Animal] Rex can move.
[Mammal] Rex | legs: 4 | warm-blooded
[Dog] Breed: German Shepherd | trained

--- Using methods from every level ---
Rex breathes.
Rex grows. Age: 3
Rex eats food.
Rex says: Woof woof!
Rex nurses its young with milk.
Rex fetches the ball!
Name: Rex | Breed: German Shepherd | Age: 3 years | Trained: Yes

--- Destruction (child first, then up the chain) ---
[Dog] Rex dog lifecycle ended.
[Mammal] Rex mammal lifecycle ended.
[Animal] Rex animal lifecycle ended.
[LivingThing] Rex lifecycle ended.
```

Public, Protected, and Private Inheritance

The **access specifier on the inheritance line** controls how the parent's members are seen in the child and by outside code.

```
class Child : [ACCESS] Parent
```

HOW PARENT MEMBERS ARE SEEN IN THE CHILD			
Parent member	public inherit	protected inherit	private
public	public	protected	private
protected	protected	protected	private
private	inaccessible	inaccessible	inaccessible

```cpp
#include <iostream>
#include <string>

class Base {
public:
    int pubData    = 1;
    void pubFunc() { std::cout << "Base::pubFunc\n"; }

protected:
    int protData   = 2;
    void protFunc(){ std::cout << "Base::protFunc\n"; }

private:
    int privData   = 3;   // NEVER accessible in derived, regardless
    void privFunc(){ std::cout << "Base::privFunc\n"; }
};

// ─── PUBLIC INHERITANCE: IS-A relationship ──────────────────────────
// Use this for: Dog IS-A Animal, Car IS-A Vehicle
// Parent's public stays public → outside code can use it
class PublicChild : public Base {
public:
    void test() {
        pubData  = 10;   // ✓ public  → public  in child
        protData = 20;   // ✓ protected → protected in child
        // privData = 30; ← ✗ always inaccessible
    }
};

// ─── PROTECTED INHERITANCE: has-a style, with inheritance ───────────
// Use this for: implementation using internals of base, not IS-A
// Parent's public becomes protected → outside code CANNOT use it
class ProtectedChild : protected Base {
public:
    void test() {
        pubData  = 10;   // ✓ public  → protected in child
        protData = 20;   // ✓ protected → protected in child
    }
};

// ─── PRIVATE INHERITANCE: strongest form of hiding ──────────────────
// Use this for: "implemented-in-terms-of" (an alternative to composition)
// Parent's public becomes private → outside code CANNOT use it
```

```cpp
// Grandchild classes ALSO cannot use it
class PrivateChild : private Base {
public:
    void test() {
        pubData  = 10;   // ✓ public  → private in child
        protData = 20;   // ✓ protected → private in child
    }
    // Expose selectively with 'using':
    using Base::pubFunc;   // Re-exposes just this one method
};

int main() {
    std::cout << "=== Inheritance Access Levels ===\n\n";

    PublicChild   pc;
    ProtectedChild tc;
    PrivateChild  vc;

    // Public inheritance: outside code CAN use parent's public members
    pc.pubData = 5;    // ✓ still public
    pc.pubFunc();      // ✓ still public

    // Protected inheritance: outside code CANNOT use parent's public members
    // tc.pubData = 5; // ✗ compile error! now protected
    // tc.pubFunc();   // ✗ compile error!

    // Private inheritance: outside code CANNOT use parent's public members
    // vc.pubData = 5; // ✗ compile error! now private
    vc.pubFunc();      // ✓ re-exposed via 'using'

    std::cout << "Access specifier demo complete.\n";
    return 0;
}
```

When to Use Each

CHOOSING YOUR INHERITANCE ACCESS LEVEL	
public	IS-A relationship. Dog IS-A Animal. Use 99% of the time. Parent interface fully available outside.
protected	Rare. Implementation inheritance where you want grandchildren to still have access.
private	"Implemented-in-terms-of". An alternative to composition. The world sees no relationship. Stack implemented using a private Vector.

Constructors in Inheritance

Understanding exactly when and how constructors fire across an inheritance chain is essential for safe C++.

```
CONSTRUCTION ORDER:
_____________________

1. Base class constructor (outermost ancestor first)
2. Member constructors (in declaration order)
3. Derived class constructor body

DESTRUCTION ORDER (always reverse):
_____________________________________

1. Derived class destructor
2. Member destructors (reverse declaration order)
3. Base class destructor
```

```cpp
#include <iostream>
#include <string>

class Engine {
public:
    int horsepower;
    Engine(int hp) : horsepower(hp) {
        std::cout << "  [Engine] Constructed: " << hp << " HP\n";
    }
    ~Engine() {
        std::cout << "  [Engine] Destroyed\n";
    }
};

class Vehicle {
protected:
    std::string make;
    int         year;
public:
    Vehicle(const std::string& m, int y) : make(m), year(y) {
        std::cout << "  [Vehicle] Constructed: " << year << " " << make << "\n";
    }
    virtual ~Vehicle() {
        std::cout << "  [Vehicle] Destroyed: " << make << "\n";
    }
};

class Car : public Vehicle {
private:
    Engine engine;     // member object — constructed AFTER Vehicle base
    int    numDoors;
public:
    // Initializer list: Vehicle first, then Engine member, then numDoors
    Car(const std::string& mk, int yr, int hp, int doors)
        : Vehicle(mk, yr),       // 1. Base class
          engine(hp),            // 2. Member objects (in declaration order)
          numDoors(doors)        // 3. Other members
    {
        // 4. Body runs last
        std::cout << "  [Car] Constructed: " << numDoors << " doors\n";
    }
    ~Car() override {
        std::cout << "  [Car] Destroyed\n";
```

```
        // After this:  engine destructor, then Vehicle destructor
    }

    void display() const {
        std::cout << year << " " << make << " | "
                  << engine.horsepower << " HP | "
                  << numDoors << " doors\n";
    }
};

class ElectricCar : public Car {
private:
    double batteryCapacity;   // kWh
public:
    ElectricCar(const std::string& mk, int yr, int hp,
                int doors, double battery)
        : Car(mk, yr, hp, doors),     // Calls Car constructor
          batteryCapacity(battery)
    {
        std::cout << "  [ElectricCar] Constructed: "
                  << battery << " kWh battery\n";
    }
    ~ElectricCar() override {
        std::cout << "  [ElectricCar] Destroyed\n";
    }
    void displayRange() const {
        std::cout << "Estimated range: " << (batteryCapacity * 6.0)
                  << " km\n";
    }
};

int main() {
    std::cout << "=== Constructors in Inheritance ===\n\n";

    std::cout << "--- Constructing ElectricCar ---\n";
    ElectricCar tesla("Tesla", 2024, 670, 4, 100.0);

    std::cout << "\n";
    tesla.display();
    tesla.displayRange();

    std::cout << "\n--- Destructors (reverse order) ---\n";
```

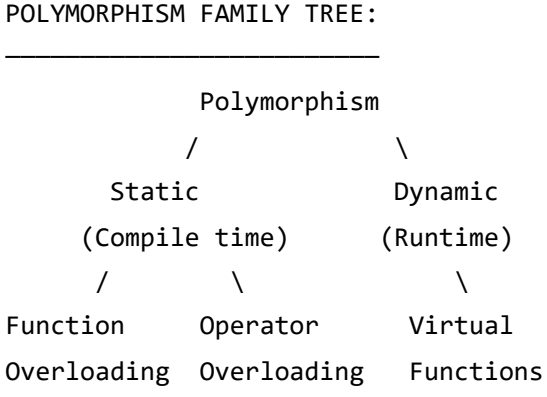
```
    return 0;
}
```

Output:

```
=== Constructors in Inheritance ===

--- Constructing ElectricCar ---
  [Vehicle] Constructed: 2024 Tesla
  [Engine] Constructed: 670 HP
  [Car] Constructed: 4 doors
  [ElectricCar] Constructed: 100 kWh battery

2024 Tesla | 670 HP | 4 doors
Estimated range: 600 km

--- Destructors (reverse order) ---
  [ElectricCar] Destroyed
  [Car] Destroyed
  [Engine] Destroyed
  [Vehicle] Destroyed: Tesla
```

15.2 Introduction to Polymorphism

Polymorphism — "many forms" — lets the same code behave differently depending on the actual type of the object it operates on. There are two kinds: **static** (resolved at compile time) and **dynamic** (resolved at runtime).



Static Polymorphism

Static polymorphism is resolved entirely at **compile time**. The compiler decides which function to call based on the types visible in the code. No runtime overhead. This includes **function overloading** and **templates**.

```cpp
#include <iostream>
#include <string>
#include <cmath>

// ─── STATIC POLYMORPHISM: FUNCTION OVERLOADING ──────────────────────
// Same function name, different parameter types.
// The compiler picks the right one at compile time.

class MathUtils {
public:
    // Three versions of area — compiler selects based on argument types
    static double area(double radius) {
        // Circle
        return M_PI * radius * radius;
    }

    static double area(double width, double height) {
        // Rectangle
        return width * height;
    }

    static double area(double base, double height, bool /*triangle*/) {
        // Triangle (extra bool just to distinguish from rectangle)
        return 0.5 * base * height;
    }

    // Overloading on type:
    static void print(int    val) { std::cout << "int:    " << val    << "\n"; }
    static void print(double val) { std::cout << "double: " << val    << "\n"; }
    static void print(const std::string& val)
                                  { std::cout << "string: " << val    << "\n"; }
};

// ─── STATIC POLYMORPHISM: TEMPLATES ─────────────────────────────────
// Template functions generate different compiled functions per type.
// Zero runtime overhead — all done at compile time.
template <typename T>
T maxOf(T a, T b) {
    return (a > b) ? a : b;
}

template <typename T>
void printPair(T a, T b) {
```

```
    std::cout << "Pair: (" << a << ", " << b
              << ") | max: " << maxOf(a, b) << "\n";
}

int main() {
    std::cout << "=== Static Polymorphism ===\n\n";

    std::cout << "-- Function overloading --\n";
    std::cout << "Circle area (r=5):          "
              << MathUtils::area(5.0) << "\n";
    std::cout << "Rectangle area (4x6):       "
              << MathUtils::area(4.0, 6.0) << "\n";
    std::cout << "Triangle area (base=3,h=8): "
              << MathUtils::area(3.0, 8.0, true) << "\n";

    std::cout << "\n-- Overloading on type --\n";
    MathUtils::print(42);
    MathUtils::print(3.14);
    MathUtils::print(std::string("hello"));

    std::cout << "\n-- Template polymorphism --\n";
    printPair(3, 7);           // int version generated
    printPair(2.5, 1.8);       // double version generated
    printPair(std::string("apple"), std::string("mango")); // string version

    return 0;
}
```

Dynamic Polymorphism

Dynamic polymorphism is resolved at **runtime** through virtual functions and base class pointers/references. The program doesn't know at compile time which function to call — it decides based on the actual object type.

DYNAMIC POLYMORPHISM — RUNTIME DISPATCH:

```
Animal* ptr;          ← compile time: "this is an Animal pointer"
ptr = new Dog();      ← runtime: "but it actually holds a Dog!"

ptr->makeSound();     ← Which makeSound() runs?
                        At compile time: unknown.
                        At runtime: Dog::makeSound() — because the
                        actual object is a Dog. VTable routes the call.
```

```cpp
#include <iostream>
#include <string>
#include <vector>

// ─── BASE CLASS ──────────────────────────────────────────────────────
class Shape {
protected:
    std::string color;
public:
    Shape(const std::string& c) : color(c) {}
    virtual ~Shape() = default;

    // Virtual functions enable dynamic dispatch
    virtual double area()      const = 0;   // Pure virtual
    virtual double perimeter() const = 0;   // Pure virtual
    virtual void   draw()      const {      // Virtual (has default)
        std::cout << "Drawing a " << color << " shape.\n";
    }

    // Non-virtual: same for all shapes
    void describe() const {
        std::cout << color << " shape | area="
                  << area() << " | perimeter=" << perimeter() << "\n";
    }
};

class Circle : public Shape {
    double radius;
public:
    Circle(const std::string& c, double r) : Shape(c), radius(r) {}
    double area()      const override { return M_PI * radius * radius; }
    double perimeter() const override { return 2.0 * M_PI * radius; }
    void   draw()      const override {
        std::cout << "Drawing " << color << " circle (r=" << radius << ")\n";
    }
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(const std::string& c, double w, double h)
        : Shape(c), w(w), h(h) {}
    double area()      const override { return w * h; }
```

```
    double perimeter() const override { return 2.0 * (w + h); }
    void   draw()      const override {
        std::cout << "Drawing " << color
                  << " rectangle (" << w << "x" << h << ")\n";
    }
};

class Triangle : public Shape {
    double a, b, c;   // three sides
public:
    Triangle(const std::string& col, double a, double b, double c)
        : Shape(col), a(a), b(b), c(c) {}
    double perimeter() const override { return a + b + c; }
    double area() const override {
        double s = perimeter() / 2.0;
        return std::sqrt(s * (s-a) * (s-b) * (s-c));  // Heron's formula
    }
    void draw() const override {
        std::cout << "Drawing " << color
                  << " triangle (sides: " << a << ", " << b << ", " << c << ")\n";
    }
};

// ── POLYMORPHIC FUNCTIONS ──────────────────────────────────────
// These work on ANY Shape — current or future!
double totalArea(const std::vector<Shape*>& shapes) {
    double total = 0.0;
    for (const auto* s : shapes) total += s->area();
    return total;
}

void drawAll(const std::vector<Shape*>& shapes) {
    for (const auto* s : shapes) s->draw();   // Virtual dispatch!
}

int main() {
    std::cout << "=== Dynamic Polymorphism ===\n\n";

    std::vector<Shape*> canvas;
    canvas.push_back(new Circle("red",    5.0));
    canvas.push_back(new Rectangle("blue", 4.0, 6.0));
    canvas.push_back(new Triangle("green", 3.0, 4.0, 5.0));
    canvas.push_back(new Circle("yellow", 2.5));
```

```
    std::cout << "-- Drawing all shapes --\n";
    drawAll(canvas);   // Dynamic dispatch: correct draw() for each

    std::cout << "\n-- Describing all shapes --\n";
    for (const auto* s : canvas) s->describe();

    std::cout << "\n-- Total canvas area: "
              << totalArea(canvas) << "\n";

    // Cleanup
    for (auto* s : canvas) delete s;
    return 0;
}
```

16.0 Function Overloading and Operator Overloading

🎯 Learning Objectives

- Overload functions to handle different types and argument counts cleanly
- Understand the rules that govern function overload resolution
- Overload operators to make user-defined types behave like built-ins
- Implement `+`, `=`, `+=`, `==`, `<<`, `>>`, `new`, and assignment operators

Introduction: Natural Syntax for Your Types

```
WITHOUT overloading:                WITH overloading:
____________________________        ____________________________

Vector v3 = addVectors(v1, v2);     Vector v3 = v1 + v2;
printVector(v);                     std::cout << v;
copyVector(v2, v1);                 v2 = v1;
if (equalVectors(v1, v2)) ...       if (v1 == v2) ...

The overloaded version reads exactly like math — natural and intuitive.
```

16.1 Function Overloading

Function overloading allows multiple functions with the **same name** but **different parameter lists**. The compiler selects the best match.

Overload Resolution Rules

```
HOW THE COMPILER PICKS:
_______________________

1. Exact match              ← preferred first
2. Trivial conversions      (int → const int)
3. Numeric promotion        (char → int, float → double)
4. Standard conversions     (int → double, double → int)
5. User-defined conversions (your conversion operators)
6. Varargs (...)            ← last resort

If two candidates are equally good → AMBIGUITY ERROR
```

```cpp
#include <iostream>
#include <string>
#include <cmath>

// ─── OVERLOADING: different parameter TYPES ──────────────────────────
class Logger {
private:
    std::string prefix;
public:
    Logger(const std::string& p) : prefix(p) {}

    // Same name 'log' — four different overloads
    void log(int val) const {
        std::cout << "[" << prefix << "] INT:    " << val << "\n";
    }
    void log(double val) const {
        std::cout << "[" << prefix << "] DOUBLE: " << val << "\n";
    }
    void log(const std::string& msg) const {
        std::cout << "[" << prefix << "] STRING: " << msg << "\n";
    }
    void log(bool val) const {
        std::cout << "[" << prefix << "] BOOL:   "
                  << (val ? "true" : "false") << "\n";
    }
};

// ─── OVERLOADING: different number of parameters ─────────────────────
class Geometry {
public:
    // Volume: 1 param = sphere, 2 = cylinder, 3 = box
    static double volume(double radius) {
        // Sphere: (4/3)πr³
        return (4.0 / 3.0) * M_PI * radius * radius * radius;
    }
    static double volume(double radius, double height) {
        // Cylinder: πr²h
        return M_PI * radius * radius * height;
    }
    static double volume(double length, double width, double height) {
        // Box: l×w×h
        return length * width * height;
    }
```

```
    // Power: 1 param = square, 2 = nth power
    static double power(double base) { return base * base; }
    static double power(double base, int exp) {
        return std::pow(base, exp);
    }
};

// ─── DEFAULT PARAMETERS vs OVERLOADING ──────────────────────────────
// Tip: when trailing args have natural defaults, use default params instead
class Timer {
    double duration;
    std::string unit;
public:
    // Default parameter approach — often cleaner than overloads
    void set(double d, const std::string& u = "seconds") {
        duration = d;
        unit = u;
        std::cout << "Timer set: " << duration << " " << unit << "\n";
    }
};

int main() {
    std::cout << "=== Function Overloading Demo ===\n\n";

    Logger lg("APP");
    lg.log(42);
    lg.log(3.14159);
    lg.log(std::string("System started"));
    lg.log(true);

    std::cout << "\n-- Geometry volume overloads --\n";
    std::cout << "Sphere (r=3):       "
              << Geometry::volume(3.0) << "\n";
    std::cout << "Cylinder (r=2,h=5): "
              << Geometry::volume(2.0, 5.0) << "\n";
    std::cout << "Box (4x3x2):        "
              << Geometry::volume(4.0, 3.0, 2.0) << "\n";

    std::cout << "\n-- Power overloads --\n";
    std::cout << "5 squared:          " << Geometry::power(5.0) << "\n";
    std::cout << "2 to the 10th:      " << Geometry::power(2.0, 10) << "\n";
```

```
    return 0;
}
```

What You Cannot Overload On

```
✗ Return type ALONE: int f(); vs double f();  — same signature, ambiguous
✗ const alone on non-pointer/reference params:
    void f(int x);   and   void f(const int x);  — treated identically
✓ const on member function itself IS different:
    void display();          ← can be called on non-const objects
    void display() const;    ← can be called on const objects
```

16.2 Operator Overloading

C++ lets you redefine how operators (`+` , `==` , `<<` , etc.) work on your custom types, making them feel like built-in types.

The Rules

```
OPERATOR OVERLOADING RULES:
_____________________________

✓ Can overload: + - * / % ^ & | ~ ! = < > += -= == != <= >= ++ --
                <<  >>  []  ()  ->  new  delete  and many more
✗ Cannot overload: ::  .  .*  ?:  sizeof  typeid
✗ Cannot invent new operators: e.g., ** for exponentiation
✓ Cannot change arity: unary stays unary, binary stays binary
✓ At least one operand must be a user-defined type
✓ Cannot change precedence or associativity

SYNTAX CHOICES:
_________________

Member function:   T operator+(const T& other) const;
  → left operand is 'this'
  → suitable for = += -= *= [] ()

Friend function:   friend T operator+(const T& lhs, const T& rhs);
  → neither operand is 'this'
  → suitable for + - * / == != << >>
  → enables: 3 + myObj (left operand can be non-class type)
```

Complete Example: Vector2D Class

```
#include <iostream>
#include <cmath>
#include <stdexcept>

class Vector2D {
private:
    double x;
    double y;

public:
    // ── CONSTRUCTORS ──────────────────────────────────────────
    Vector2D() : x(0.0), y(0.0) {}
    Vector2D(double x, double y) : x(x), y(y) {}

    // Getters
    double getX() const { return x; }
    double getY() const { return y; }
    double magnitude() const { return std::sqrt(x*x + y*y); }

    // ── operator+ : addition (friend — no 'this' required) ────
    // v3 = v1 + v2;
    friend Vector2D operator+(const Vector2D& lhs, const Vector2D& rhs) {
        return Vector2D(lhs.x + rhs.x, lhs.y + rhs.y);
    }

    // ── operator- : subtraction ───────────────────────────────
    friend Vector2D operator-(const Vector2D& lhs, const Vector2D& rhs) {
        return Vector2D(lhs.x - rhs.x, lhs.y - rhs.y);
    }

    // ── operator* : scalar multiplication ─────────────────────
    // v2 = v1 * 3.0;
    friend Vector2D operator*(const Vector2D& v, double scalar) {
        return Vector2D(v.x * scalar, v.y * scalar);
    }
    // Also: 3.0 * v1 (scalar on the LEFT)
    friend Vector2D operator*(double scalar, const Vector2D& v) {
        return Vector2D(v.x * scalar, v.y * scalar);
    }

    // ── operator= : copy assignment (member — 'this' is left side) —
```

```
// v2 = v1;
Vector2D& operator=(const Vector2D& other) {
    if (this == &other) return *this;   // Self-assignment guard!
    x = other.x;
    y = other.y;
    return *this;   // Return *this to enable chaining: a = b = c;
}

// ── operator+= : compound addition ──────────────────────────
// v1 += v2;  equivalent to v1 = v1 + v2;
Vector2D& operator+=(const Vector2D& other) {
    x += other.x;
    y += other.y;
    return *this;
}

// ── operator-= : compound subtraction ───────────────────────
Vector2D& operator-=(const Vector2D& other) {
    x -= other.x;
    y -= other.y;
    return *this;
}

// ── operator== : equality comparison ────────────────────────
// if (v1 == v2)
friend bool operator==(const Vector2D& lhs, const Vector2D& rhs) {
    const double eps = 1e-9;   // Floating-point epsilon
    return (std::abs(lhs.x - rhs.x) < eps &&
            std::abs(lhs.y - rhs.y) < eps);
}

// ── operator!= : inequality (implement in terms of ==) ──────
friend bool operator!=(const Vector2D& lhs, const Vector2D& rhs) {
    return !(lhs == rhs);
}

// ── operator< : less-than (by magnitude — enables sorting) ──
friend bool operator<(const Vector2D& lhs, const Vector2D& rhs) {
    return lhs.magnitude() < rhs.magnitude();
}

// ── operator- : unary negation ──────────────────────────────
// v2 = -v1;
```

```
    Vector2D operator-() const {
        return Vector2D(-x, -y);
    }

    // ── operator++ : prefix increment ──────────────────────────
    // ++v;  (adds 1 to both components)
    Vector2D& operator++() {
        x += 1.0; y += 1.0;
        return *this;
    }

    // ── operator++ : postfix increment (takes dummy int) ───────
    // v++;
    Vector2D operator++(int) {
        Vector2D temp = *this;   // Save current state
        x += 1.0; y += 1.0;     // Increment
        return temp;             // Return old state
    }

    // ── operator<< : stream output (friend — left side is ostream) ─
    // std::cout << v;
    friend std::ostream& operator<<(std::ostream& os, const Vector2D& v) {
        os << "(" << v.x << ", " << v.y << ")";
        return os;   // Return os to enable chaining: cout << v1 << v2;
    }

    // ── operator>> : stream input ──────────────────────────────
    // std::cin >> v;
    friend std::istream& operator>>(std::istream& is, Vector2D& v) {
        is >> v.x >> v.y;
        return is;
    }
};

int main() {
    std::cout << "=== Operator Overloading — Vector2D ===\n\n";

    Vector2D v1(3.0, 4.0);
    Vector2D v2(1.0, 2.0);

    std::cout << "v1 = " << v1 << "\n";
    std::cout << "v2 = " << v2 << "\n";
    std::cout << "|v1| = " << v1.magnitude() << "\n\n";
```

```
// operator+
Vector2D v3 = v1 + v2;
std::cout << "v1 + v2 = " << v3 << "\n";

// operator-
std::cout << "v1 - v2 = " << (v1 - v2) << "\n";

// operator* (scalar)
std::cout << "v1 * 3  = " << (v1 * 3.0) << "\n";
std::cout << "2 * v2  = " << (2.0 * v2) << "\n";

// operator= (assignment)
Vector2D v4;
v4 = v1;
std::cout << "\nv4 = v1 → " << v4 << "\n";

// operator+=
v4 += v2;
std::cout << "v4 += v2 → " << v4 << "\n";

// operator== and !=
std::cout << "\nv1 == v1: " << (v1 == v1 ? "true" : "false") << "\n";
std::cout << "v1 == v2: " << (v1 == v2 ? "true" : "false") << "\n";
std::cout << "v1 != v2: " << (v1 != v2 ? "true" : "false") << "\n";

// Unary -
std::cout << "\n-v1 = " << (-v1) << "\n";

// Prefix and postfix ++
Vector2D v5(1.0, 1.0);
std::cout << "\nv5 = " << v5 << "\n";
std::cout << "++v5 = " << (++v5) << " | v5 = " << v5 << "\n";
std::cout << "v5++ = " << (v5++) << " | v5 = " << v5 << "\n";

// Chaining with <<
std::cout << "\nChained: " << v1 << " + " << v2 << " = " << (v1 + v2) << "\n";

// operator<< input: uncomment to test
// Vector2D v6;
// std::cout << "Enter a vector (x y): ";
// std::cin >> v6;
// std::cout << "You entered: " << v6 << "\n";
```

```
    return 0;
}
```

Output:

```
=== Operator Overloading — Vector2D ===

v1 = (3, 4)
v2 = (1, 2)
|v1| = 5

v1 + v2 = (4, 6)
v1 - v2 = (2, 2)
v1 * 3  = (9, 12)
2 * v2  = (2, 4)

v4 = v1 → (3, 4)
v4 += v2 → (4, 6)

v1 == v1: true
v1 == v2: false
v1 != v2: true

-v1 = (-3, -4)

v5 = (1, 1)
++v5 = (2, 2) | v5 = (2, 2)
v5++ = (2, 2) | v5 = (3, 3)

Chained: (3, 4) + (1, 2) = (4, 6)
```

Overloading `new` and `delete`

```
#include <iostream>
#include <cstdlib>

class TrackedObject {
private:
    int id;
    static int allocationCount;

public:
    TrackedObject(int i) : id(i) {
        std::cout << "  TrackedObject #" << id << " constructed.\n";
    }
    ~TrackedObject() {
        std::cout << "  TrackedObject #" << id << " destroyed.\n";
    }

    // ── operator new: custom allocation ──────────────────────
    // Called instead of default ::new
    static void* operator new(std::size_t size) {
        allocationCount++;
        std::cout << "  [Custom new] Allocating " << size
                  << " bytes. Total allocations: " << allocationCount << "\n";
        void* ptr = std::malloc(size);
        if (!ptr) throw std::bad_alloc();
        return ptr;
    }

    // ── operator delete: custom deallocation ─────────────────
    // Called instead of default ::delete
    static void operator delete(void* ptr) noexcept {
        allocationCount--;
        std::cout << "  [Custom delete] Freeing. Remaining: "
                  << allocationCount << "\n";
        std::free(ptr);
    }

    // ── operator new[]: array allocation ─────────────────────
    static void* operator new[](std::size_t size) {
        std::cout << "  [Custom new[]] Allocating array: "
                  << size << " bytes\n";
```

```
        return std::malloc(size);
    }

    static void operator delete[](void* ptr) noexcept {
        std::cout << "  [Custom delete[]] Freeing array.\n";
        std::free(ptr);
    }

    static int getAllocationCount() { return allocationCount; }
};

int TrackedObject::allocationCount = 0;

int main() {
    std::cout << "=== Custom new/delete Demo ===\n\n";

    std::cout << "-- Single object --\n";
    TrackedObject* t1 = new TrackedObject(1);  // calls operator new
    TrackedObject* t2 = new TrackedObject(2);
    delete t1;                                  // calls operator delete
    delete t2;

    std::cout << "\n-- Array of objects --\n";
    TrackedObject* arr = new TrackedObject[3]{TrackedObject(10),
                                              TrackedObject(11),
                                              TrackedObject(12)};
    delete[] arr;

    std::cout << "\nFinal allocation count: "
              << TrackedObject::getAllocationCount() << "\n";
    return 0;
}
```

Operator Overloading — Comparison Table

OPERATOR OVERLOADING QUICK REFERENCE		
Operator	Member or Friend?	Notes
=	MUST member	Return *this; self-assign check needed
[]	MUST member	Return reference for lvalue use
()	MUST member	Functor / callable objects
->	MUST member	Smart pointer pattern
+= -=	member	Return *this
*= /=	member	Modifies left operand
++ --	member	prefix: T&; postfix: T (int dummy)
+ -	friend	Return new object (not *this)
* /	friend	Symmetric — either side can be scalar
== !=	friend	Return bool; != in terms of ==
< >	friend	Return bool; define < then get > free
<< >>	MUST friend	Left side is ostream/istream

Task 3

Implement a `Matrix` class (2D, square, NxN) with the following operator overloads:

1. `operator+` — matrix addition
2. `operator*` — matrix multiplication (not element-wise)
3. `operator==` — equality check (element-wise comparison)
4. `operator=` — deep copy assignment
5. `operator[]` — row access: `m[i]` returns a pointer to row `i`
6. `operator<<` — pretty-print the matrix

Bonus: Add `operator*` for scalar multiplication: `Matrix m2 = m1 * 2.5;`

17.0 Virtual Functions

🎯 Learning Objectives

- Understand the difference between early (static) and late (dynamic) binding
- Implement virtual functions and understand the vtable mechanism
- Design abstract base classes using pure virtual functions
- Recognize the diamond problem and solve it with virtual inheritance
- Know when to use virtual functions and their performance cost

Introduction: The Need for Runtime Decision-Making

Consider a drawing application. You have a `Canvas` holding an array of `Shape*` pointers — circles, rectangles, triangles. When you call `draw()` on each pointer, which version should run?

Without virtual functions: the compiler always calls `Shape::draw()` — wrong!
With virtual functions: the runtime looks up the actual object type and calls the correct override — right!

```
WITHOUT virtual:                    WITH virtual:
────────────────                    ─────────────
Shape* s = new Circle();            Shape* s = new Circle();
s->draw();                          s->draw();
// ALWAYS calls Shape::draw()       // Calls Circle::draw() — correct!
// Ignores the fact it's            // Runtime decides based on
// actually a Circle                // actual object type
```

17.1 Early Binding vs Late Binding

Definition of Binding

Binding is the process of connecting a function call (the name in code) to the actual function body that will execute. Every function call must be bound to an address in memory.

```
SOURCE CODE:            COMPILED CODE:
____________            ______________

animal->speak();        CALL ???         ← where does this go?
                                         ↑ Binding answers this question.
```

Early (Static) Binding

In **early binding** (also called **static binding**), the compiler resolves which function to call **at compile time**, based solely on the declared type of the variable — not the actual runtime type.

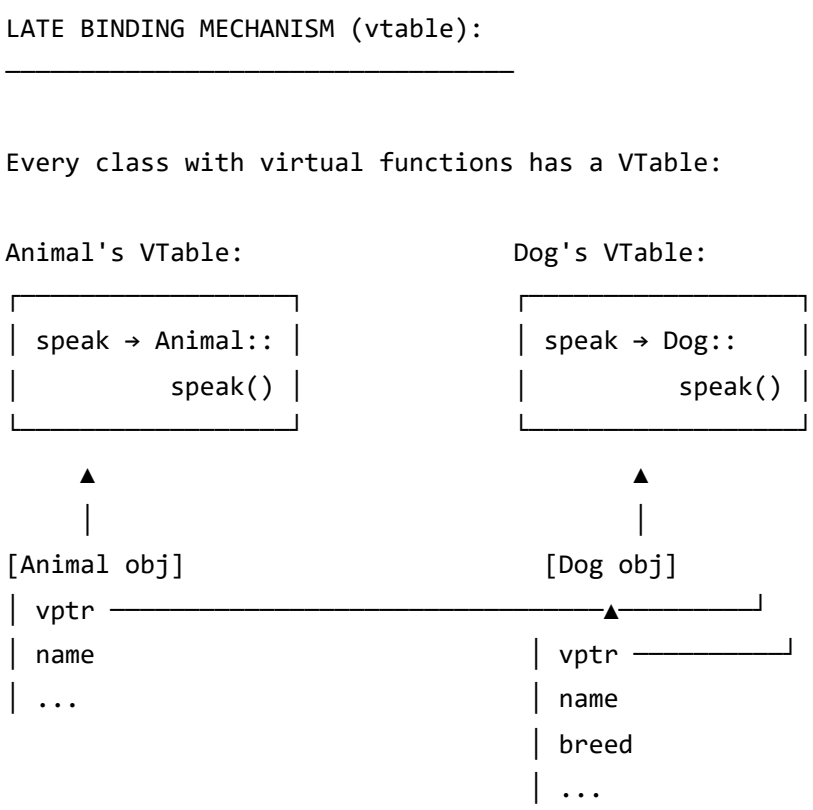
```
EARLY BINDING MECHANISM:
________________________

Animal* ptr = new Dog();   ← compiler sees type: Animal*
ptr->speak();              ← compile time: "call Animal::speak"
                             (ignores that ptr actually holds a Dog)

The address of Animal::speak() is baked directly into the
compiled instruction. Fast! But inflexible.
```

```cpp
#include <iostream>
#include <string>

// ─── EARLY BINDING (no virtual) ──────────────────────────────────────
class AnimalNV {   // NV = No Virtual
protected:
    std::string name;
public:
    AnimalNV(const std::string& n) : name(n) {}

    // NO virtual keyword — always early binding
    void speak() const {
        std::cout << name << " (Animal): generic sound\n";
    }
    void describe() const {
        std::cout << "I am " << name << " of type Animal\n";
    }
};

class DogNV : public AnimalNV {
public:
    DogNV(const std::string& n) : AnimalNV(n) {}

    // This shadows parent's speak() but does NOT override (no virtual)
    void speak() const {
        std::cout << name << " (Dog): Woof!\n";
    }
};

void earlyBindingDemo() {
    std::cout << "--- Early Binding (no virtual) ---\n";

    DogNV   d("Rex");
    AnimalNV& ref = d;       // reference to DogNV, but declared as AnimalNV&

    d.speak();     // Direct call on DogNV → DogNV::speak()  (Rex (Dog): Woof!)
    ref.speak();   // Through AnimalNV& → AnimalNV::speak()  (Rex (Animal): generic)
    // ^ Both compile-time decisions based on DECLARED type, not actual object!

    AnimalNV* ptr = new DogNV("Buddy");
    ptr->speak();  // AnimalNV::speak() — WRONG! We wanted DogNV::speak().
    delete ptr;
```

```
    std::cout << "Early binding: compiler decided at compile time.\n\n";
}
```

Late (Dynamic) Binding

In **late binding** (also called **dynamic binding**), the decision of which function to call is made at **runtime**, based on the actual type of the object. This requires the `virtual` keyword.



```cpp
#include <iostream>
#include <string>
#include <memory>

// ─── LATE BINDING (with virtual) ──────────────────────────────────
class Animal {
protected:
    std::string name;
public:
    Animal(const std::string& n) : name(n) {}
    virtual ~Animal() = default;

    // virtual → late binding. Decision made at runtime.
    virtual void speak() const {
        std::cout << name << " (Animal): generic sound\n";
    }

    virtual std::string getType() const { return "Animal"; }

    void describe() const {
        // describe() is non-virtual, but it CALLS virtual speak()
        // So describe() itself uses early binding for its own call,
        // but speak() inside it still uses late binding!
        std::cout << "I am " << name << ": ";
        speak();   // virtual call — resolves at runtime to actual type!
    }
};

class Dog : public Animal {
    std::string breed;
public:
    Dog(const std::string& n, const std::string& b)
        : Animal(n), breed(b) {}

    void speak() const override {
        std::cout << name << " (Dog, " << breed << "): Woof!\n";
    }
    std::string getType() const override { return "Dog"; }
};

class Cat : public Animal {
public:
    Cat(const std::string& n) : Animal(n) {}
```

```
    void speak() const override {
        std::cout << name << " (Cat): Meow!\n";
    }
    std::string getType() const override { return "Cat"; }
};

class Parrot : public Animal {
    std::string phrase;
public:
    Parrot(const std::string& n, const std::string& p)
        : Animal(n), phrase(p) {}
    void speak() const override {
        std::cout << name << " (Parrot): " << phrase << "!\n";
    }
    std::string getType() const override { return "Parrot"; }
};

void lateBindingDemo() {
    std::cout << "--- Late Binding (with virtual) ---\n";

    // Array of base-class pointers holding different derived objects
    Animal* animals[4];
    animals[0] = new Dog("Rex", "German Shepherd");
    animals[1] = new Cat("Whiskers");
    animals[2] = new Parrot("Polly", "Pretty bird");
    animals[3] = new Dog("Buddy", "Labrador");

    // ONE loop, FOUR different behaviors — this is polymorphism!
    for (int i = 0; i < 4; i++) {
        std::cout << "[" << animals[i]->getType() << "] ";
        animals[i]->speak();   // virtual: correct version per actual type
    }

    std::cout << "\n-- describe() calling virtual speak() internally --\n";
    for (int i = 0; i < 4; i++)
        animals[i]->describe();

    for (int i = 0; i < 4; i++) delete animals[i];
}

int main() {
    std::cout << "=== Early vs Late Binding ===\n\n";
    earlyBindingDemo();
```

```
    lateBindingDemo();
    return 0;
}
```

Output:

```
--- Early Binding (no virtual) ---
Rex (Dog): Woof!
Rex (Animal): generic sound         ← bound to Animal::speak at compile time
Buddy (Animal): generic sound       ← same issue with pointer
Early binding: compiler decided at compile time.

--- Late Binding (with virtual) ---
[Dog] Rex (Dog, German Shepherd): Woof!
[Cat] Whiskers (Cat): Meow!
[Parrot] Polly (Parrot): Pretty bird!
[Dog] Buddy (Dog, Labrador): Woof!

-- describe() calling virtual speak() internally --
I am Rex: Rex (Dog, German Shepherd): Woof!      ← Dog::speak via virtual!
I am Whiskers: Whiskers (Cat): Meow!
I am Polly: Polly (Parrot): Pretty bird!
I am Buddy: Buddy (Dog, Labrador): Woof!
```

Comparing Early and Late Binding

```
┌──────────────────────────────────────────────────────────────────┐
│              EARLY vs LATE BINDING COMPARISON                    │
├──────────────────────┬──────────────────┬────────────────────────┤
│  Aspect              │  Early (Static)  │  Late (Dynamic)        │
├──────────────────────┼──────────────────┼────────────────────────┤
│  Resolution time     │  Compile time    │  Runtime               │
│  Keyword needed      │  (default)       │  virtual               │
│  Performance         │  Faster          │  Tiny overhead (vtable)│
│  Memory overhead     │  None            │  vptr per object + vtable │
│  Flexibility         │  Fixed           │  Adapts to actual type │
│  Use case            │  Non-polymorphic │  Polymorphic hierarchies │
│  Inlineable?         │  Yes             │  Rarely (needs object) │
│  Called on           │  Declared type   │  Actual object type    │
└──────────────────────┴──────────────────┴────────────────────────┘
```

```
Performance numbers (modern CPU):
  Direct (early) call:  ~1 ns
  Virtual (late) call:  ~3-5 ns (extra pointer dereferences)
  For most apps: this difference is negligible.
  For hot loops with millions of calls: profile first.
```

17.2 Virtual Functions — Deep Dive

Basics of Virtual Functions

A **virtual function** is a member function declared with the `virtual` keyword in a base class. When called through a base class pointer or reference, the actual function called depends on the **runtime type** of the object.

Syntax and Rules

```
class Base {
public:
    // 1. virtual declaration in base
    virtual void func();

    // 2. Pure virtual (abstract) — child MUST override
    virtual void mustOverride() = 0;

    // 3. Virtual destructor — CRITICAL for correct deletion
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    // 4. override keyword — compile-time check that we're overriding
    void func() override;

    // 5. final — prevents further overriding
    void mustOverride() override final;
};

class Further : public Derived {
    // void mustOverride() override;  ← COMPILE ERROR: it's final!
};
```

The `override` and `final` Keywords

```
override:
_________
Tells the compiler: "I intend to override a virtual function."
If the base has no matching virtual → compile error. Catches typos!

WITHOUT override:
void Speak() const { ... }    // silently DOESN'T override speak()
                              // (capital S is a new function!)
WITH override:
void Speak() const override { ... }  // ERROR: no virtual Speak() in base
                                     // Caught at compile time ✓

final:
______
On a class:    class Leaf final { };         // No further inheritance
On a method:   void func() override final;   // No further overriding
```

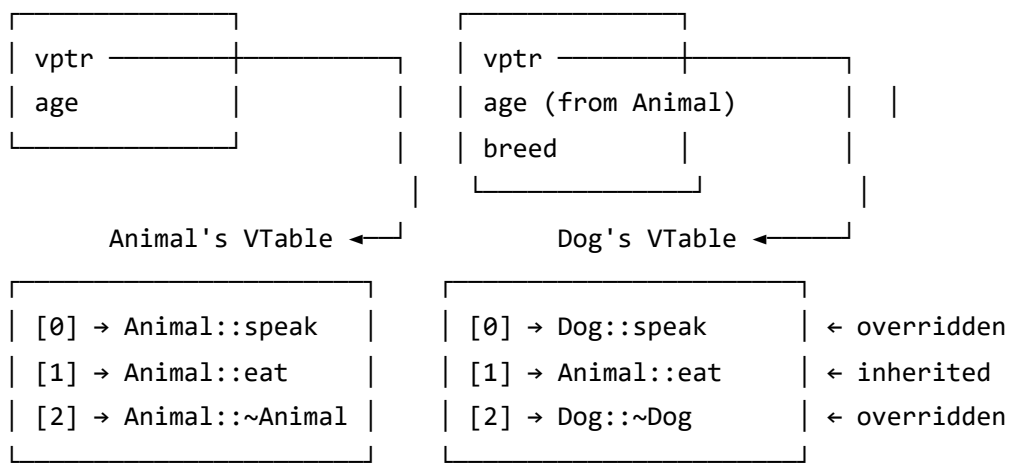
Virtual Function Table (VTable) and VPtr

Every class with virtual functions has a **vtable** — a static array of function pointers. Every object of such a class has a hidden **vptr** — a pointer to its class's vtable.

MEMORY LAYOUT WITH VIRTUAL FUNCTIONS:
_____

```
class Animal {              class Dog : public Animal {
  virtual void speak();       void speak() override;
  virtual void eat();         void fetch();        // not virtual
  int age;                    std::string breed;
};                          };
```

Animal object layout:



Dog object layout:

When you call: ptr->speak(); (where ptr is Animal* holding a Dog)

1. Load ptr (address of Dog object)
2. Load vptr from object (points to Dog's VTable)
3. Load entry [0] from VTable → address of Dog::speak
4. Jump to Dog::speak

Total: ~2 extra memory reads vs a direct call. Very fast!

Proving VTable Behavior

```cpp
#include <iostream>
#include <string>

class Instrument {
public:
    virtual ~Instrument() = default;
    virtual void play() const {
        std::cout << "Instrument: playing...\n";
    }
    virtual std::string name() const { return "Instrument"; }
};

class Guitar : public Instrument {
    int numStrings;
public:
    Guitar(int s = 6) : numStrings(s) {}
    void play() const override {
        std::cout << "Guitar (" << numStrings << " strings): Strum!\n";
    }
    std::string name() const override { return "Guitar"; }
};

class Piano : public Instrument {
public:
    void play() const override {
        std::cout << "Piano: Classical melody...\n";
    }
    std::string name() const override { return "Piano"; }
};

class ElectricGuitar : public Guitar {
    std::string effect;
public:
    ElectricGuitar(const std::string& e)
        : Guitar(6), effect(e) {}
    void play() const override {
        std::cout << "Electric Guitar [" << effect << "]: SHRED!\n";
    }
    std::string name() const override { return "Electric Guitar"; }
};
```

```cpp
// Demonstrates virtual dispatch — same code, different results
void performConcert(Instrument** lineup, int count) {
    std::cout << "\n=== Concert ===\n";
    for (int i = 0; i < count; i++) {
        std::cout << "[" << lineup[i]->name() << "] ";
        lineup[i]->play();   // virtual dispatch: correct play() every time
    }
}

// Shows slicing — the danger of passing by value to base class
void slicingDemo(Instrument byValue) {   // ← passed by VALUE to base
    std::cout << "[sliced] ";
    byValue.play();   // Always Instrument::play() — the derived part is GONE!
}

int main() {
    std::cout << "=== Virtual Functions Deep Dive ===\n";

    Instrument* lineup[4];
    lineup[0] = new Guitar(6);
    lineup[1] = new Piano();
    lineup[2] = new ElectricGuitar("distortion");
    lineup[3] = new Guitar(12);

    performConcert(lineup, 4);

    std::cout << "\n--- Object Slicing Danger ---\n";
    Guitar g(6);
    slicingDemo(g);   // Sliced: only Instrument part copied, Guitar part lost

    // Always use pointer or reference for polymorphism — never by value!
    Instrument& ref = g;
    std::cout << "[ref, no slicing] "; ref.play();   // Guitar::play() ✓

    for (int i = 0; i < 4; i++) delete lineup[i];
    return 0;
}
```

Object Slicing — The Hidden Danger

```
OBJECT SLICING:
_________________

void badFunc(Animal a) {    ← takes Animal by VALUE
    a.speak();              ← always Animal::speak(), even if you passed a Dog!
}

Dog d("Rex", "Lab");
badFunc(d);    ← Dog is COPIED into Animal parameter.
                  The Dog-specific data (breed, etc.) is SLICED OFF.
                  Only the Animal portion is copied.

FIX: always use pointer or reference for polymorphism:
void goodFunc(Animal* a) { a->speak(); }    // pointer — no slicing
void goodFunc(Animal& a) { a.speak(); }     // reference — no slicing
```

Best Practices and Common Pitfalls

```
BEST PRACTICES:
✓ Always declare base class destructor as virtual
  (without it: deleting a Dog* through Animal* won't call Dog's destructor)
✓ Use 'override' on every function that overrides a virtual
✓ Use 'final' when you know a class/method should not be extended further
✓ Avoid calling virtual functions in constructors/destructors
  (the vtable is not fully set up yet — calls go to the current class, not derived)
✓ Pass polymorphic objects by pointer or reference (never by value)
✓ Prefer pure virtual for functions that MUST be overridden

COMMON PITFALLS:
❌ Non-virtual base destructor → derived destructor not called → memory leak
❌ Object slicing: Animal a = Dog();  → Dog portion lost
❌ Calling virtual in constructor: base version called, not derived (surprising!)
❌ Forgetting 'override': silent shadowing instead of overriding
❌ Massive vtable in deep hierarchies: profile if performance-critical
```

17.3 Pure Virtual Functions and Abstract Base Classes

Pure Virtual Functions

A **pure virtual function** has no implementation in the base class. It **forces every concrete derived class** to provide its own implementation.

```
virtual void draw() = 0;   // = 0 makes it pure virtual
```

```
PURE VIRTUAL:
_____________
class Shape {
    virtual double area() = 0;    ← pure virtual: no body in Shape
};

Shape s;        ← COMPILE ERROR: Shape is abstract (has pure virtuals)
Shape* s = new Shape();  ← COMPILE ERROR same reason

class Circle : public Shape {
    double area() override { return M_PI * r * r; }   // MUST provide this
};
Circle c;       ← OK: Circle provides all pure virtual implementations
```

Abstract Base Classes

A class with **at least one pure virtual function** is an **abstract class**. It cannot be instantiated — it exists purely as a contract (interface) that derived classes must fulfil.

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <cmath>

// ─── ABSTRACT BASE CLASS: defines the interface ──────────────────────
class Drawable {
public:
    virtual ~Drawable() = default;

    // Pure virtual: every drawable thing MUST implement these
    virtual void draw()   const = 0;
    virtual void resize(double factor) = 0;
    virtual double area() const = 0;

    // Non-pure virtual: default behavior, but can be overridden
    virtual std::string getDescription() const {
        return "Drawable object (area=" + std::to_string(area()) + ")";
    }

    // Non-virtual: shared utility — not overridable
    void highlight() const {
        std::cout << "*** HIGHLIGHTED: ";
        draw();
        std::cout << " ***\n";
    }
};

// ─── CONCRETE CLASSES: implement all pure virtuals ──────────────────
class Circle : public Drawable {
    double x, y, radius;
public:
    Circle(double x, double y, double r) : x(x), y(y), radius(r) {}

    void draw() const override {
        std::cout << "Circle at (" << x << "," << y
                  << ") radius=" << radius;
    }
    void resize(double factor) override {
        radius *= factor;
        std::cout << "Circle resized to radius=" << radius << "\n";
    }
```

```
    double area() const override {
        return M_PI * radius * radius;
    }
    std::string getDescription() const override {
        return "Circle (r=" + std::to_string(radius) + ")";
    }
};

class Rectangle : public Drawable {
    double x, y, w, h;
public:
    Rectangle(double x, double y, double w, double h)
        : x(x), y(y), w(w), h(h) {}

    void draw() const override {
        std::cout << "Rectangle at (" << x << "," << y
                  << ") " << w << "x" << h;
    }
    void resize(double factor) override {
        w *= factor; h *= factor;
        std::cout << "Rectangle resized to " << w << "x" << h << "\n";
    }
    double area() const override { return w * h; }
};

class Triangle : public Drawable {
    double base, height, x, y;
public:
    Triangle(double x, double y, double b, double h)
        : base(b), height(h), x(x), y(y) {}

    void draw() const override {
        std::cout << "Triangle at (" << x << "," << y
                  << ") base=" << base << " h=" << height;
    }
    void resize(double factor) override {
        base *= factor; height *= factor;
        std::cout << "Triangle resized to base=" << base << "\n";
    }
    double area() const override { return 0.5 * base * height; }
};

// ── CANVAS: works with any Drawable — past, present, or FUTURE ──────
```

```
class Canvas {
    std::vector<std::unique_ptr<Drawable>> objects;

public:
    void add(std::unique_ptr<Drawable> obj) {
        objects.push_back(std::move(obj));
    }

    void drawAll() const {
        std::cout << "\n--- Canvas: drawing all ---\n";
        for (const auto& obj : objects) {
            obj->draw();
            std::cout << "\n";
        }
    }

    void resizeAll(double factor) {
        std::cout << "\n--- Resizing all by x" << factor << " ---\n";
        for (auto& obj : objects) obj->resize(factor);
    }

    double totalArea() const {
        double total = 0.0;
        for (const auto& obj : objects) total += obj->area();
        return total;
    }

    void describeAll() const {
        std::cout << "\n--- Descriptions ---\n";
        for (const auto& obj : objects)
            std::cout << "  " << obj->getDescription() << "\n";
    }
};

int main() {
    std::cout << "=== Abstract Base Class Demo ===\n";

    // Drawable d;  ← COMPILE ERROR: Drawable is abstract!

    Canvas canvas;
    canvas.add(std::make_unique<Circle>(10.0, 10.0, 5.0));
    canvas.add(std::make_unique<Rectangle>(0.0, 0.0, 8.0, 4.0));
    canvas.add(std::make_unique<Triangle>(5.0, 5.0, 6.0, 9.0));
```

```
    canvas.add(std::make_unique<Circle>(20.0, 20.0, 3.0));

    canvas.drawAll();
    std::cout << "\nTotal area: " << canvas.totalArea() << "\n";

    canvas.resizeAll(2.0);
    canvas.drawAll();
    std::cout << "\nTotal area after resize: " << canvas.totalArea() << "\n";

    canvas.describeAll();

    return 0;
}
```

Interfaces in C++ Using Abstract Classes

C++ has no `interface` keyword (unlike Java/C#). We simulate interfaces using **abstract classes where ALL functions are pure virtual**.

```cpp
#include <iostream>
#include <string>

// ─── INTERFACE: Serializable ──────────────────────────────────────
// All functions pure virtual → this is a pure interface
class Serializable {
public:
    virtual ~Serializable() = default;
    virtual std::string serialize()            const = 0;
    virtual void        deserialize(const std::string& data) = 0;
};

// ─── INTERFACE: Printable ─────────────────────────────────────────
class Printable {
public:
    virtual ~Printable() = default;
    virtual void print() const = 0;
    virtual void printToFile(const std::string& filename) const = 0;
};

// ─── INTERFACE: Comparable ────────────────────────────────────────
class Comparable {
public:
    virtual ~Comparable() = default;
    virtual int compareTo(const Comparable& other) const = 0;
    // compareTo returns: negative if less, 0 if equal, positive if greater
};

// ─── CONCRETE CLASS: implements multiple interfaces ──────────────
class Employee : public Serializable,
                 public Printable,
                 public Comparable {
private:
    std::string name;
    std::string id;
    double      salary;

public:
    Employee(const std::string& n, const std::string& i, double s)
        : name(n), id(i), salary(s) {}

    // ── Serializable interface ──────────────────────────
    std::string serialize() const override {
```

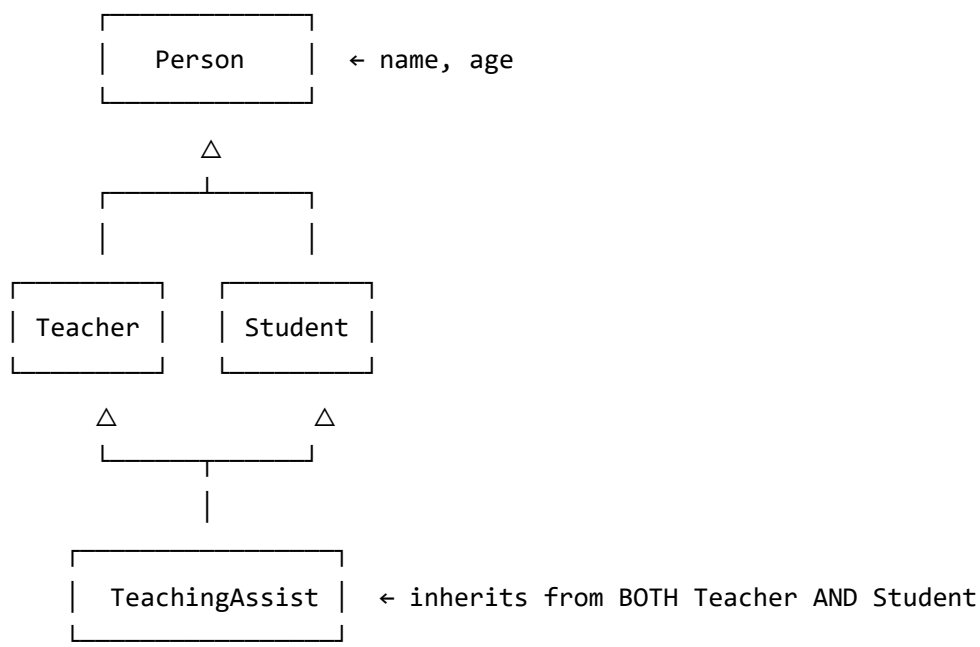
```
        return id + "," + name + "," + std::to_string(salary);
    }
    void deserialize(const std::string& data) override {
        // Simple CSV parsing
        size_t p1 = data.find(',');
        size_t p2 = data.find(',', p1 + 1);
        id     = data.substr(0, p1);
        name   = data.substr(p1 + 1, p2 - p1 - 1);
        salary = std::stod(data.substr(p2 + 1));
    }

    // ── Printable interface ──────────────────────────────
    void print() const override {
        std::cout << "[Employee] ID:" << id
                  << " | " << name
                  << " | $" << salary << "\n";
    }
    void printToFile(const std::string& fn) const override {
        std::cout << "[Writing to " << fn << "]: " << serialize() << "\n";
    }

    // ── Comparable interface ─────────────────────────────
    int compareTo(const Comparable& other) const override {
        const Employee& otherEmp = dynamic_cast<const Employee&>(other);
        if (salary < otherEmp.salary)  return -1;
        if (salary > otherEmp.salary)  return  1;
        return 0;
    }

    double getSalary() const { return salary; }
};

int main() {
    std::cout << "=== Interfaces via Abstract Classes ===\n\n";

    Employee e1("Alice Smith",   "E001", 75000.0);
    Employee e2("Bob Johnson",   "E002", 90000.0);
    Employee e3("Carol Williams","E003", 75000.0);

    // Use via interface pointers — multiple interfaces!
    Printable*    p  = &e1;
    Serializable* s  = &e1;
```

```
    p->print();
    std::cout << "Serialized: " << s->serialize() << "\n";

    // Round-trip: serialize then deserialize
    std::string data = e2.serialize();
    std::cout << "\nSerialized e2: " << data << "\n";
    Employee e4("", "", 0.0);
    e4.deserialize(data);
    std::cout << "Deserialized:  "; e4.print();

    // Comparable
    std::cout << "\ne1 vs e2 salary: "
              << (e1.compareTo(e2) < 0 ? "e1 lower" :
                  e1.compareTo(e2) > 0 ? "e1 higher" : "equal")
              << "\n";
    std::cout << "e1 vs e3 salary: "
              << (e1.compareTo(e3) == 0 ? "equal" : "different")
              << "\n";

    return 0;
}
```

Diamond Problem and Virtual Inheritance

The **diamond problem** occurs in multiple inheritance when a class inherits from two classes that share a common ancestor. Without virtual inheritance, the grandparent is duplicated — causing ambiguity and wasted memory.

THE DIAMOND:



WITHOUT virtual inheritance:
TeachingAssistant has TWO copies of Person!
- One via Teacher → Person
- One via Student → Person
ta.name = ? ← AMBIGUOUS! Which Person::name?

WITH virtual inheritance:
Only ONE shared copy of Person exists.
ta.name works fine — no ambiguity.

```cpp
#include <iostream>
#include <string>

// ─── WITHOUT virtual inheritance (shows the problem) ──────────────────
namespace WithoutVirtual {

struct A { int x = 1; void show() { std::cout << "A::x = " << x << "\n"; } };
struct B : A { };    // B has one A
struct C : A { };    // C has one A
struct D : B, C { }; // D has TWO A's: one from B, one from C

void demo() {
    D d;
    // d.x = 5;       // ← COMPILE ERROR: ambiguous! B::A::x or C::A::x?
    d.B::x = 5;    // Must qualify explicitly
    d.C::x = 10;   // Different x!
    d.B::show();   // Shows 5
    d.C::show();   // Shows 10  (two separate Person copies!)
    std::cout << "Two separate copies of A in D.\n";
}

} // namespace

// ─── WITH virtual inheritance (the fix) ──────────────────────────────
namespace WithVirtual {

struct A { int x = 1; void show() { std::cout << "A::x = " << x << "\n"; } };
struct B : virtual A { };    // virtual: share A
struct C : virtual A { };    // virtual: share A
struct D : B, C { };         // D has exactly ONE A

void demo() {
    D d;
    d.x = 42;    // ✓ No ambiguity! Only ONE x.
    d.show();    // Shows 42
    std::cout << "Only ONE copy of A in D.\n";
}

} // namespace

// ─── REAL-WORLD DIAMOND: Academic Staff Example ──────────────────────
class Person {
protected:
```

```
    std::string name;
    int         age;
public:
    Person(const std::string& n, int a) : name(n), age(a) {
        std::cout << "  [Person] " << name << " constructed.\n";
    }
    virtual ~Person() = default;
    virtual void introduce() const {
        std::cout << "Hi, I'm " << name << ", age " << age << ".\n";
    }
};

class Teacher : virtual public Person {     // virtual inheritance!
protected:
    std::string subject;
    int         yearsExp;
public:
    Teacher(const std::string& n, int a,
            const std::string& sub, int exp)
        : Person(n, a), subject(sub), yearsExp(exp) {
        std::cout << "  [Teacher] " << name
                  << " teaches " << subject << ".\n";
    }
    virtual void teach() const {
        std::cout << name << " teaches " << subject
                  << " (" << yearsExp << " yrs exp).\n";
    }
};

class Student : virtual public Person {     // virtual inheritance!
protected:
    std::string major;
    double      gpa;
public:
    Student(const std::string& n, int a,
            const std::string& maj, double g)
        : Person(n, a), major(maj), gpa(g) {
        std::cout << "  [Student] " << name
                  << " studies " << major << ".\n";
    }
    virtual void study() const {
        std::cout << name << " studies " << major
                  << " (GPA: " << gpa << ").\n";
```

```
    }
};

// TeachingAssistant inherits from BOTH Teacher and Student
// Because both use virtual inheritance, only ONE Person base exists
class TeachingAssistant : public Teacher, public Student {
    std::string researchArea;
public:
    TeachingAssistant(const std::string& n, int a,
                      const std::string& sub, int exp,
                      const std::string& maj, double g,
                      const std::string& research)
        // TA constructor must explicitly initialize the virtual base (Person)!
        : Person(n, a),
          Teacher(n, a, sub, exp),
          Student(n, a, maj, g),
          researchArea(research)
    {
        std::cout << "  [TA] " << name
                  << " researches " << researchArea << ".\n";
    }

    void introduce() const override {
        std::cout << "I'm " << name
                  << ", a TA teaching " << subject
                  << " and studying " << major << ".\n";
    }

    void doWork() const {
        teach();   // from Teacher
        study();   // from Student
        std::cout << name << " also researches " << researchArea << ".\n";
    }
};

int main() {
    std::cout << "=== Diamond Problem Demo ===\n\n";

    std::cout << "--- Without virtual inheritance ---\n";
    WithoutVirtual::demo();

    std::cout << "\n--- With virtual inheritance ---\n";
    WithVirtual::demo();
```

```
    std::cout << "\n--- Real-world Diamond: TeachingAssistant ---\n";
    std::cout << "Constructing TA:\n";
    TeachingAssistant ta("Dr. Chen", 30,
                         "Algorithms", 2,
                         "Computer Science", 3.9,
                         "Machine Learning");

    std::cout << "\n";
    ta.introduce();
    ta.doWork();

    // ONE name — no ambiguity:
    std::cout << "\nta.name (single shared Person): ";
    ta.introduce();   // works without qualification

    return 0;
}
```

Output:

```
=== Diamond Problem Demo ===

--- Without virtual inheritance ---
Two separate copies of A in D.

--- With virtual inheritance ---
A::x = 42
Only ONE copy of A in D.

--- Real-world Diamond: TeachingAssistant ---
Constructing TA:
  [Person] Dr. Chen constructed.        ← Only ONE Person!
  [Teacher] Dr. Chen teaches Algorithms.
  [Student] Dr. Chen studies Computer Science.
  [TA] Dr. Chen researches Machine Learning.

I'm Dr. Chen, a TA teaching Algorithms and studying Computer Science.
Dr. Chen teaches Algorithms (2 yrs exp).
Dr. Chen studies Computer Science (GPA: 3.9).
Dr. Chen also researches Machine Learning.

ta.name (single shared Person):
I'm Dr. Chen, a TA teaching Algorithms and studying Computer Science.
```

Complete Virtual Function Summary

VIRTUAL FUNCTIONS COMPLETE REFERENCE	
Declaration	Effect
virtual void f()	Enables override. Default impl provided.
virtual void f() =0	Pure virtual. No impl. Class is abstract.
void f() override	Verify overriding (compile-time check).
void f() final	Override this one last time; no more.
virtual ~Base()	Ensures correct destruction via base ptr.
VTABLE COST:	
• One vtable per class (not per object) — shared static data	
• One vptr per object — 8 bytes on 64-bit systems	
• One extra indirection per virtual call — usually negligible	
VIRTUAL INHERITANCE (diamond fix):	
class B : virtual public A { ... };	
• Only ONE copy of A regardless of how many paths to it	
• Most-derived class must explicitly init the virtual base	
• Slight extra overhead: one more pointer per virtual base	

⚠️ Common Mistakes — Chapters 15–17

```
// ❌ MISTAKE 1: Non-virtual base destructor → memory leak
class Animal { ~Animal() {} };        // NOT virtual
class Dog : public Animal { char* data; ~Dog() { delete[] data; } };
Animal* a = new Dog("Rex");
delete a;   // Only Animal::~Animal() called! Dog::~Dog() SKIPPED → leak!
// FIX: always virtual ~Animal() {}

// ❌ MISTAKE 2: Calling virtual function in constructor
class Base {
    Base() { init(); }               // init() is virtual
    virtual void init() { std::cout << "Base::init\n"; }
};
class Derived : public Base {
    void init() override { std::cout << "Derived::init\n"; }
};
Derived d;    // Prints "Base::init" — NOT "Derived::init"!
// During Base ctor, vtable is Base's, not Derived's. Always.

// ❌ MISTAKE 3: Object slicing
void process(Animal a) { a.speak(); }  // Takes by value!
Dog d("Rex");
process(d);    // Dog portion sliced! Always speaks as Animal.
// FIX: void process(Animal& a) or void process(Animal* a)

// ❌ MISTAKE 4: Forgetting 'override'
class Base  { virtual void func() const; };
class Child : public Base {
    void func()       { }  // Missing const → NOT an override! New function.
    // With 'override': void func() override { }  → compile error caught!
};

// ❌ MISTAKE 5: Ambiguity in multiple inheritance without virtual
class A { public: int x; };
class B : public A { };
class C : public A { };
class D : public B, public C { };
D d; d.x = 5;  // ERROR: which A::x? B's or C's?
// FIX: class B : virtual public A; class C : virtual public A;
```

📝 Final Exercises — Chapters 15–17

Exercise 1 (Beginner — Inheritance): Create a `Person → Employee → Manager` chain. Person has name/age. Employee adds salary/department. Manager adds teamSize and `giveBonus(percent)`. Demonstrate constructor chaining and method overriding.

Exercise 2 (Intermediate — Polymorphism + Overloading): Implement a `Calculator` class with overloaded `calculate(int, int)`, `calculate(double, double)`, and `calculate(string, string)` (string concatenation). Then add a `ScientificCalculator` that overrides with advanced math.

Exercise 3 (Advanced — Virtual + Abstract):
Design a plugin system using abstract classes:

- `Plugin` abstract base: `virtual void load() = 0`, `virtual void unload() = 0`, `virtual string getName() = 0`, `virtual void execute(string command) = 0`
- Implement: `LoggingPlugin`, `DatabasePlugin`, `NetworkPlugin`
- Create a `PluginManager` that holds `vector<Plugin*>` and can `loadAll()`, `executeAll(command)`, `unloadAll()`

🏆 Chapters 15–17 Master Quick Reference

```
╔═══════════════════════════════════════════════════════════════════╗
║          INHERITANCE + POLYMORPHISM + VIRTUAL — CHEAT SHEET        ║
╠═══════════════════════════════════════════════════════════════════╣
║  INHERITANCE TYPES:                                               ║
║  Single:       class B : public A                                 ║
║  Multiple:     class C : public A, public B                       ║
║  Multilevel:   class C : public B (B inherits A)                  ║
║  Virtual base: class B : virtual public A  (diamond fix)          ║
╠═══════════════════════════════════════════════════════════════════╣
║  POLYMORPHISM:                                                    ║
║  Static:  function overloading, templates — compile time          ║
║  Dynamic: virtual functions, base pointers — runtime              ║
╠═══════════════════════════════════════════════════════════════════╣
║  VIRTUAL KEYWORDS:                                                ║
║  virtual void f()        → overridable, has default               ║
║  virtual void f() = 0    → must override; class is abstract        ║
║  void f() override       → compiler checks override is valid      ║
║  void f() final          → no further overriding allowed          ║
║  virtual ~Base()         → always in polymorphic base classes     ║
╠═══════════════════════════════════════════════════════════════════╣
║  OPERATOR OVERLOADING MUST-KNOWS:                                 ║
║  = [] () ->    → must be member                                   ║
║  << >>         → must be friend (left side is stream)             ║
║  Always return *this from = += -= etc.                            ║
║  Always guard = against self-assignment: if(this==&o) return *this ║
║  prefix ++: returns T&   postfix ++: returns T (int dummy)        ║
╠═══════════════════════════════════════════════════════════════════╣
║  BINDING:                                                         ║
║  Early = compile-time = no virtual = fast but inflexible          ║
║  Late  = runtime = virtual + base pointer = flexible + tiny cost  ║
╚═══════════════════════════════════════════════════════════════════╝
```

📚 Additional Resources

Books

- **"Effective C++" by Scott Meyers** — Items 35–40 cover virtual functions, inheritance, and polymorphism with deep insight. Essential.
- **"More Effective C++" by Scott Meyers** — Items on operators and inheritance edge cases.
- **"C++ Primer" by Lippman, Lajoie, Moo** — Best introductory textbook covering all these topics systematically.
- **"Design Patterns" by GoF** — Most patterns rely directly on virtual functions and abstract base classes.

Online

- **learncpp.com** — Chapters 18–25 cover everything in this guide with runnable examples.
- **isocpp.github.io/CppCoreGuidelines** — Official C++ Core Guidelines: C.120–C.140 cover class hierarchies.
- **en.cppreference.com** — Authoritative syntax reference for every keyword covered here.

Tools

- **Compiler Explorer (godbolt.org)** — See the actual assembly generated. Paste `virtual void f()` and observe the vtable instructions.
- **Clang** `-Woverloaded-virtual` — Warn when a function that looks like an override is not actually virtual.
- `-fsanitize=address,undefined` — Catch slicing, virtual-call-in-ctor, and inheritance bugs at runtime.

> 🏆 **Final Pro Tip:** The guiding question for virtual functions is: *"Does a pointer to the base class need to call the right version for the actual object?"* If yes → `virtual` . A useful memory aid: **Non-virtual = compile-time contract. Virtual = runtime promise.** Design your base classes with this distinction firmly in mind, and your hierarchies will be clean, correct, and extensible.

End of Part 4: Chapters 15, 16, 17
Inheritance, Polymorphism, Overloading, Virtual Functions in C++